



Sandya Mannarswamy

Welcome to another instalment of 'CodeSport'. This month we take a quick look at the problem of finding out whether a given binary tree is in fact a binary search tree. We then discuss the problem of finding the maximum and minimum in a binary search tree.

Thanks to all the readers who sent in their comments to the problems we discussed in last month's column. Congratulations to Rohit Agarwalla and Rajeev Kumar for getting the solutions to the problem correct. Last month's takeaway problem was on identifying binary search trees. A binary search tree (BST), or ordered binary tree, is a type of binary tree where the nodes are arranged in order: for each node in the tree, all elements in the left sub-tree of the node are less than or equal to the value of the node, and all the elements in the right sub-tree of the node are greater than the node. Binary search trees are used for element insertion and lookup.

Last month's problem was to determine whether a given binary tree is, in fact, a binary search tree or not. Given a plain binary tree, you have to examine the tree to determine if it meets the requirements of a binary search tree, i.e., for every node, all of the nodes in its left tree must be less than or equal to the node, and all the nodes in its right subtree must be greater than the node. The algorithm should return *true* if the binary tree meets the criteria for a binary search tree. Else it should return *false*.

What is the best possible time complexity of such an algorithm? In order to verify that every node in the tree satisfies the binary search tree property, we need to visit each node once. Hence the best possible time complexity can not be less than $O(N)$ where N is the number of nodes in the binary tree. Now how do we come up with an algorithm that verifies whether a binary tree satisfies the binary search tree property at each node?

We use the BST property that a left child of a node is less than or equal to the node, whereas the right child of a node is greater than the node. If we visit the left child first, then the node and then the right child, then these nodes will be visited in order. Such a walk on the nodes of the tree is known as 'in order tree walk'. If we print the nodes as we visit each node, such a visit will print the nodes in monotonically increasing order. Hence all we need to do to verify that a binary tree is a BST, is to visit the nodes of the tree in order and print the elements. If such a walk results in a monotonically increasing sequence of elements, then the tree is a BST.

We can easily write a recursive algorithm to visit the binary tree in order. The steps of our recursive algorithm

at each node x are as follows:

1. Make sure that node x is not NULL.
2. Recursively print the elements in the left subtree of x .
3. Print the element at node x .
4. Recursively print the elements in the right subtree of x .

We assume that each node x contains a pointer to its left child in the field 'left', a pointer to its right child in the field 'right', and the field 'key' gives the element value contained in the node x . We can assume the utility functions `left(x)`, which returns the left child, `right(x)` which gives the right child, and `key(x)` which gives the value of the element at node x .

```
inOrderTreeWalk(x)
{
    if (x is not null)
    {
        inOrderTreeWalk(left(x));
        print the value of key(x);
        inOrderTreeWalk(right(x));
    }
}
```

Given below is the pseudo code for the function *IsThisTreeBST*, which takes the root of the given binary tree as a parameter.

```
bool IsThisTreeBST(x)
{
    get S the sequence of elements returned by inOrderTreeWalk(x);

    if (S is a monotonically increasing sequence)
        Return true;
    else
        return false;
}
```

I leave it to the reader to write the code to verify that the sequence of elements returned by the `inOrderTreeWalk` is a monotonically increasing sequence or not.

This month's programming question

In this month's column, we will stay with the binary search trees. Given a BST T , how do we find the minimum and